Part - II

Greedy method :-

General method, applications - Job searvencing with deadlines, Knapsack Problem, minimum cost spanning trees, Single source Shortest path Problem.

General method :-

-> Greedy algorithms build a solution part by part, choosing the next part in such a way, that it gives an immediate benefit. This approach never reconsiders the choices taken previously. This approach mainly used to solve optimization problems.

-> Greedy method is easy to implement and quite efficient in most of the cases.

-> In many problems, it does not produce an optimal solution though it gives an approximate solution in a resonable time

A Greedy algorithm works if a problem exhibits the following two properties:

1. Greedy Choice : A Globally optimal solution can be reached at by creating 'locally optimal solution. In other words, an optimal solution can be obtained by creating "greedy" choices.

2. Optimal substructure: optimal solution contain optimal sub solutions. In other words, answers to subproblems of an optimal solution are optimal

Steps for achieving a greedy algorithm are:

1. Feasible:- Here we check whether it satisfies all possible constraints or not, to obtain atleast one solution to our problems.
2. Local optimal choice:- In this, the choice should be the optimum which is selected from the currently available
3. Unalterable:- once the decision is made, at any subsequence step that option is not altered.

General Algorithm:-

```
Algorithm Greedy (a,n)
{
  for i = 1 to n do
  {
    x = select(a);
    if feasible(x) then
    {
      solution = solution + x;
    }
  }
}
```

Job Seaquence with deadlines:-

You are given set of jobs

→ each job has a defined deadline and some profit associated with it.

→ the profit of a job is given only when the job is completed with in its deadline.

→ only one processor is available for processing all the jobs.

→ Processor takes one unit of time to complete a job

Approach to Solution:-

→ A feasible solution would be a subset of jobs where each job of the subset gets completed within its deadline

→ value of the feasible soution would be the sum of profit of all the jobs contained in the subset

→ An optimal solution of the problem would be a feasible solution which gives the maximum profit.

Greedy Algorithm:-

→ Greedy Algorithm is adopted to determine how the next job is selected for an optimal solution.

→ The greedy algorithm described below gives optimal solution to the job-seaquencing problem.

Algorithm jobsequence:

```
Sort the jobs according to profit
for i=1 to n do
    set k = min (dmax, deadline(i))
while (k>=1) do
    if timeslot[k] is empty then
        timeslot[k] = Job[i]
        break;
    end if
    set k=k-1
end while
end for
```

Problem:-

Given the jobs, their deadlines and associated profits as shown:-

jobs	j1	j2	j3	j4	j5	j6
deadlines	5	3	3	2	4	2
profits	200	180	190	300	120	100

Find the following

1. write the optimal schedule that gives maximum profit
2. Are all the jobs completed in the optimal schedule
3. what is the maximum earned profit.

Step 1:-

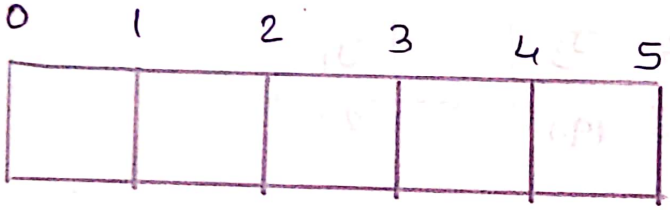
sort all the jobs in decreasing order of their profits.

jobs	j4	j1	j3	j2	j5	j6
deadlines	2	5	3	3	4	2
profits	300	200	190	180	120	100

Step 2:-

Valve of maximum deadline = 5

So, draw timeslot (or) Gantt chart with maximum time 5 units

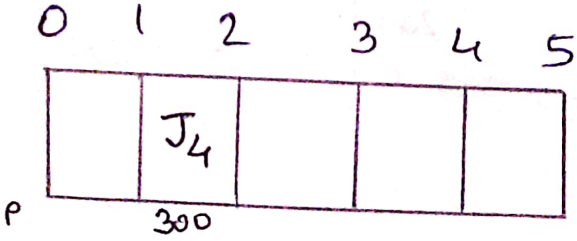


Step 3:-

Now we take job one by one in the order they appear in step-1 and we place job on timeslot as far as possible from 0..

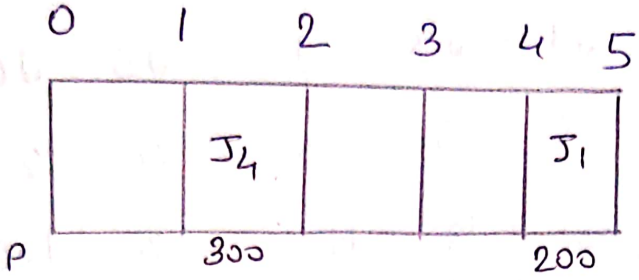
Step 3:-

we take job 'j4', since its dead line is 2, we place it in the first empty cell before deadline 2, as



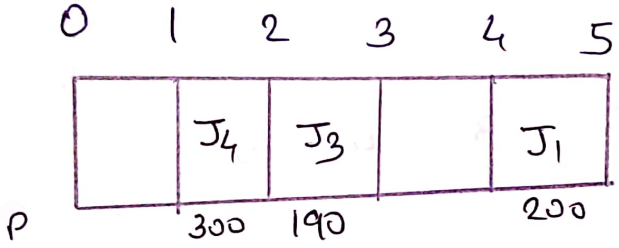
Step 4:-

Next we take job J1, since its deadline is 5, we place it in the first empty cell before dead line 5



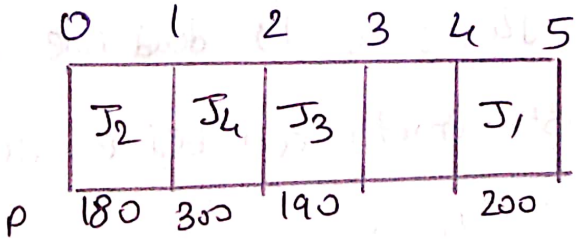
Step 5:-

we take job J3 and its deadline is 3 we place it in the first empty slot before the deading '3'.



Step 6:-

Take job J2 and its deadline is 3, so we place it in the first empty slot before deadline 3. since, second and third cells are already filled, so we place job J2 in the first cell



Step 7:-

Now we take job J5 and its dead line is 4, so, we keep it in the first empty cell before deadline 4

	0	1	2	3	4	5
	J_2	J_4	J_3	J_5	J_1	
P	180	300	190	120	200	

Now, the only job left is job 'J6' whose deadline is 2 and since all slots are filled before dead line 2 job 'J6' cannot be completed.

Now the given questions can be completed as follows

1A) The optimal schedule is J2, J4, J3, J5, J1 is the order in which the jobs must be completed in order to obtain the maximum profit

2A) All the jobs not completed in optimal schedule, because job 'J6' cannot be fit in time slot and it is not completed

3A) maximum earned profit is sum of all the jobs in optimal schedule

180 + 300 + 190 + 120 + 200 = 990 units.

Knapsack problem:-

Time complexity of job scheduling with deadlines:

$O(n^2)$

Knapsack Problem :-

We call it fractional knapsack problem using greedy approach

Fractions of items can be taken rather than having to make 0-1 choices for each item.

Fractional knapsack problem can be solvable by greedy strategy.

Steps to solve the fractional problem:-

→ compute the value per pound (profit/weight) for each item

→ Arrang in decreasing order of $\frac{\text{Profit}}{\text{weight}}$ value

→ Keep adding the objects into the knapsack (bag) until the bag is filled

→ If space is left in the knapsack and no objet left, then ignore

→ If space is left in the knapsack and any objects are left, then choose the fractional part of remaining

object cy per space left in knapsack

Algorithm Greedy knapsack

```
{
    for i=1 to n;
        compute pi/wi;
    sort object in non increasing order of p/w
    for i=1 to n
        if(m>0 && wi<=m)
            m=m-wi;
            p=p+pi;
        else
            break;
    if(m>0)
        p=p+pi(m/wi);
}
```

Time compleacity:

The total time taken for knapsack problem is O(nlogn)

Problem:-

For the given set of items and knapsack capacity 60kg. Find optimal solution using Greedy Approach

item	I_1	I_2	I_3	I_4	I_5
weight	5	10	15	22	25
Profit	30	40	45	77	90

Solution:

So given $n=5$ $w=60$ K.g

Step 1:-

Compute Profit/weight $(\frac{P}{w})$ of each item

item	I_1	I_2	I_3	I_4	I_5
Profit	30	40	45	77	90
weight	5	10	15	22	25
$\frac{P}{w}$	6	4	3	3.5	3.6

Step 2:

Sort the items in decreasing order of their P/w ration

I_1	I_2	I_5	I_4	I_3
6	4	3.6	3.5	3

Step3:-

Start filling knapsack by putting items according to $\frac{P}{w}$ ratio

Knapsack weight	Items in knapsack	Profit
60	∅	∅
60-5=55	I_1	30
55-10=45	I_1, I_2	30+40=70
45-25=20 out of 22 only 20 weight taken	I_1, I_2, I_5	70+90=160
	I_1, I_2, I_5, fraction of I_4	profit will be calculated for 20 weight $\frac{20}{22} \times 77 = 70$ Total Profit = 160+70 = 230

since ~~the~~ it is fractional knapsack problem, we can take fraction of any item

So, Knapsack will contain the following items I_1, I_2, I_5 and fraction of I_4 to get maximum profit of 230 ~~units~~.

minimum cost spanning tree:

A spanning tree is a subset of an undirected graph that has all the vertices connected by minimum number of edges.

If all the vertices are connected in a graph, then there exists atleast one spanning tree. In a graph there may exist more than one spanning tree.

Properties:-

- → A spanning tree does not have any cycle
- → Any vertex can be reached from any other vertex

minimum cost spanning tree (MST):-

It is a subset of edges of a connected weighted undirected graph that connects all the vertices together with the minimum possible total edge weight

It can be done by using

- → Prim's algorithm
- → Kruskal's algorithm

If there are 'n' number of vertices, the spanning tree should have 'n-1' number of edges.

⇒ If each edge of the graph is associated with a weight and there exist more than one spanning tree, we need to find the MST of the graph.

Prim's Algorithm (without using min heap)

```
Algorithm Prim (E, cost, n, t)
// E is the set of edges and cost is n*n adjacency matrix
// MST is computed and stored in array t[1:n-1, 1:2]
{
1. Let (K,L) be an edge of min cost in E;
                                    mincost = cost[K,L];
2. t[1,1] = K; t[1,2] = L
3. for i=1 to n do
     a. if cost[i,L] < cost[i,K] then near[i]=L;
     b. else near[i]=K;
4. near[K] = near[L]=0;
5. for i=2 to n-1
     a. let j be an index such that near[j]≠0 and
     b. cost[j, near[j]] is minimum
     c. t[i,1] = j, t[i,2]= near[j];
     d. mincost = mincost + cost[j, near[j]];
     e. near[j] = 0;
```

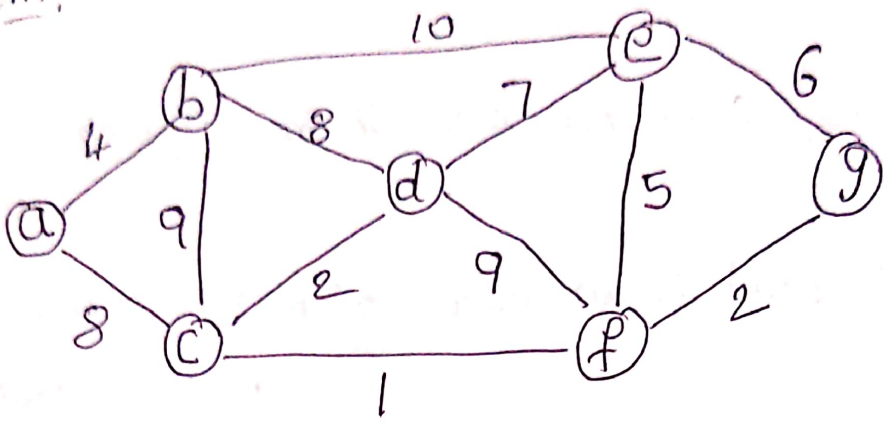
for k=1 to n do

If (near[k] ≠ 0 and cost[k, near[k]] > cost[k])

then near[k] = j

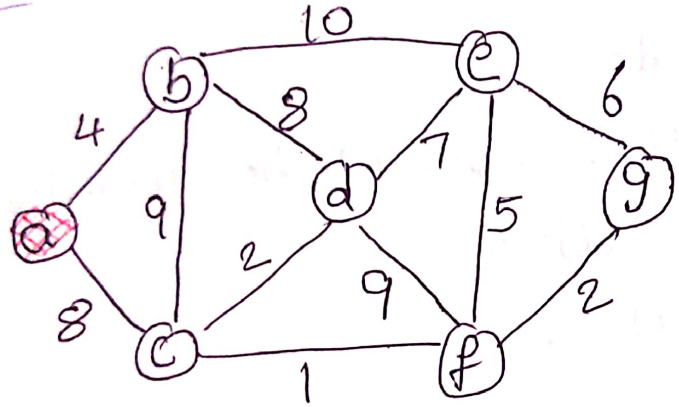
Time complexity without using min-heap is $O(n^2)$

Problem:



Solution:

①



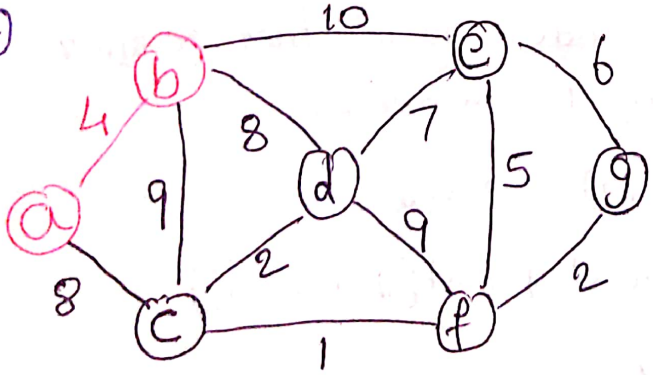
~~step 0:~~

S = {a}

V = {b, c, d, e, f, g}

lightest edge = {a, b}

②

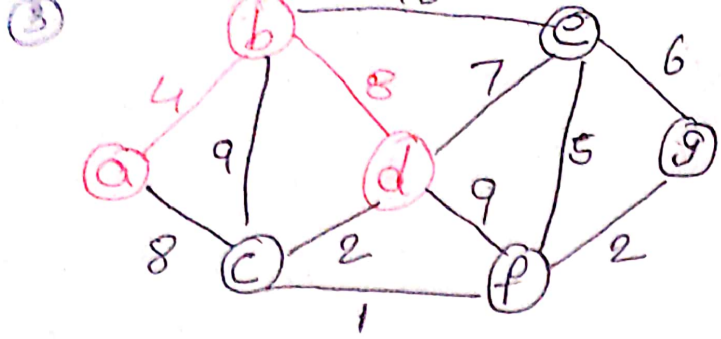


S = {a, b}

V = {c, d, e, f, g}

A = {{a, b}}

lightest edge = {b, d}, {a, c}



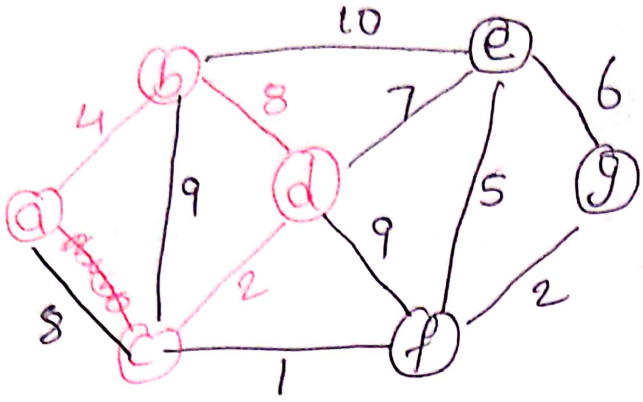
$S = \{a, b, d\}$

$V = \{c, e, f, g\}$

$A = \{\{a, b\}, \{b, d\}\}$

lightest edge $= \{d, c\}$

(4)



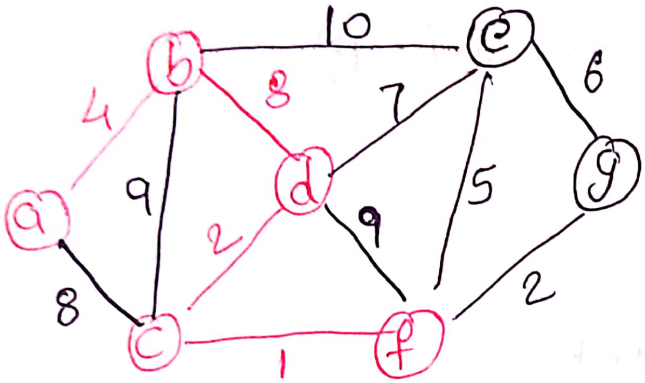
$S = \{a, b, c, d\}$

$V = \{e, f, g\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}\}$

lightest edge $= \{c, f\}$

(5)



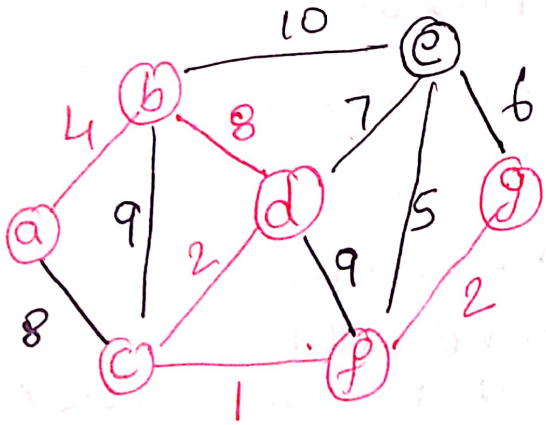
$S = \{a, b, c, d, f\}$

$V = \{e, g\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}, \{c, f\}\}$

lightest edge $= \{f, g\}$

(6)



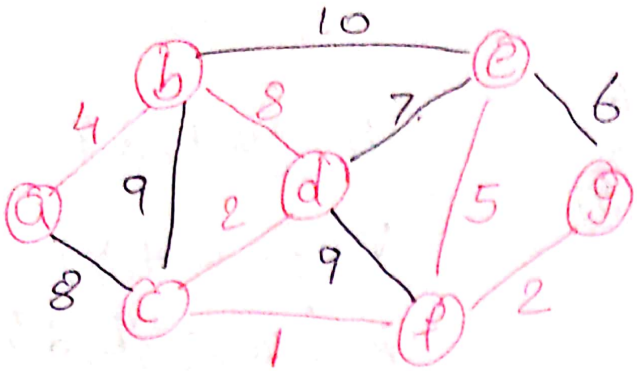
$S = \{a, b, c, d, f, g\}$

$V = \{e\}$

$A = \{\{a, b\}, \{b, d\}, \{c, d\}, \{c, f\}, \{f, g\}\}$

lightest edge $= \{f, e\}$

(7)

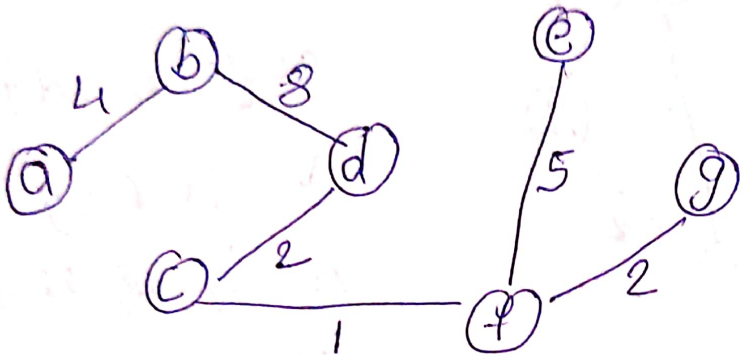


$S = \{a, b, c, d, e, f, g\}$

$V = \{ \}$

$A = \{\{a,b\}, \{b,d\}, \{c,d\}, \{c,f\}, \{f,g\}, \{f,e\}\}$

So, finally MST is



minimum cost = 24

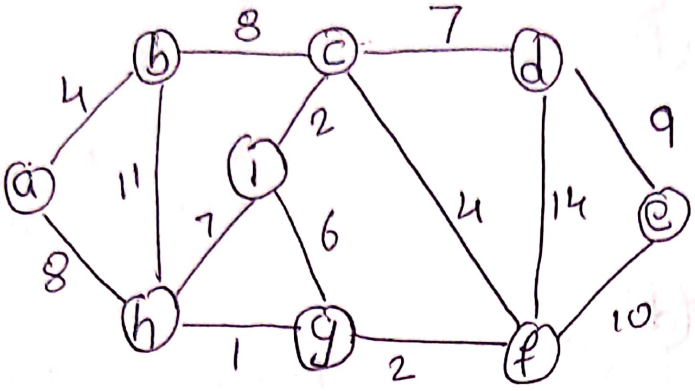
Prim's algorithm (with using min heap)

Algorithm

1. $Q \leftarrow \emptyset$
2. for each $u \in V$
   do $key[u] \leftarrow \infty$
   $\pi[u] \leftarrow NIL$
   INSERT(Q, u)
3. DECREASE_KEY$(Q, s, 0)$ → $key[s] \leftarrow 0$
4. while $Q \neq \emptyset$
   do $u \leftarrow$ EXTRACT_MIN(Q)
   for each $v \in Adj[u]$
   do if $v \in Q$ and $w(u,v) < key[v]$
   then $\pi[v] \leftarrow u$
   DECREASE_KEY$(Q, v, w(u,v))$

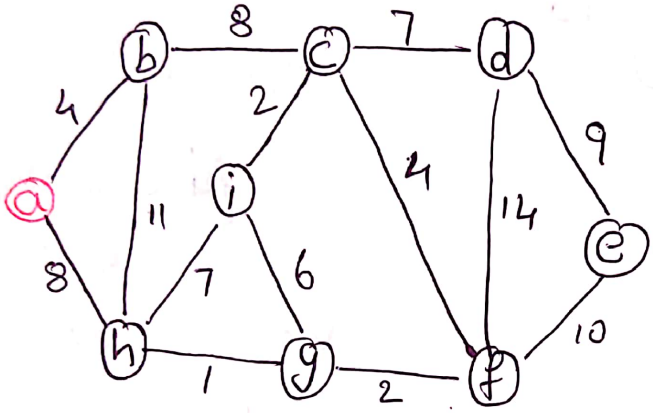
Time complexity with using min-Heap is $O(n^2 \log n)$

Problem:-



Solution:-

(1)



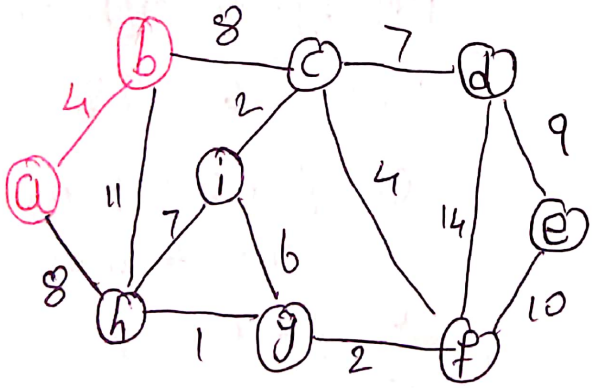
0 ∞ ∞ ∞ ∞ ∞ ∞ ∞ ∞

Q = {a, b, c, d, e, f, g, h, i}

V = ∅

Extract_min(Q) => a

(2)



key[b] = 4 $\pi(b) = a$

key[h] = 8 $\pi(h) = a$

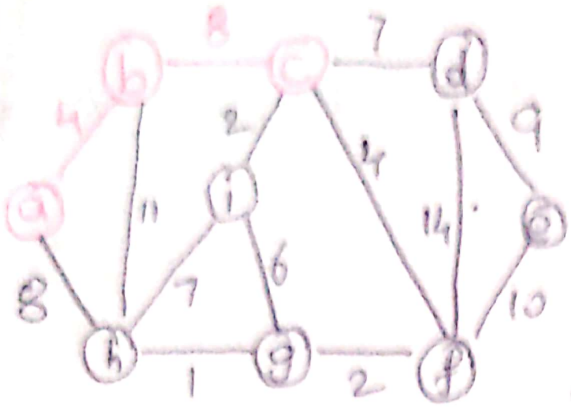
Q = {b c d e f g h i}

4 ∞ ∞ ∞ ∞ ∞ 8 ∞

V = {a}

Extract_min(Q) => b

③



Key[c] = 8 π[c] = b

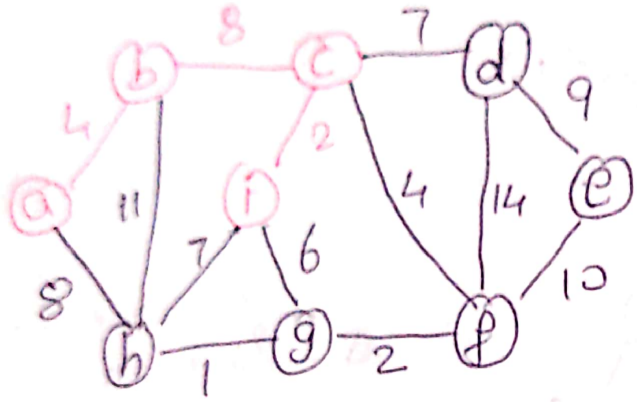
Key[h] = 8 π(h) = a
(unchanged)

Q = {c, d, e, f, g, h, i}
8 ∞ ∞ ∞ 8 ∞

V = {a, b}

Extract_min(Q) ⇒ c

④



Key[d] = 7 π[d] = c

Key[f] = 4 π[f] = c

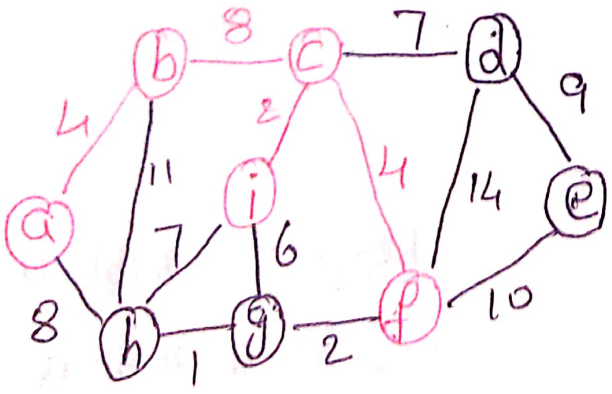
Key[i] = 2 π[i] = c

Q = {d, e, f, g, h, i}
7 ∞ 4 ∞ 8 2

V = {a, b, c}

Extract-min(Q) ⇒ i

⑤



Key[h] = 7 π[h] = i

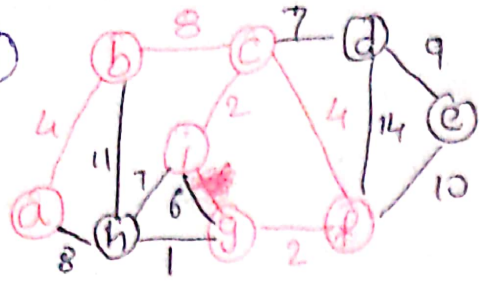
Key[g] = 6 π[g] = i

Q = {d, e, f, g, h}
7 ∞ 4 6 8

V = {a, b, c, i}

Extract-min(Q) ⇒ f

(6)



Key[g] = 2 π[g] = f

Key[d] = 7 π[d] = c (unchanged)

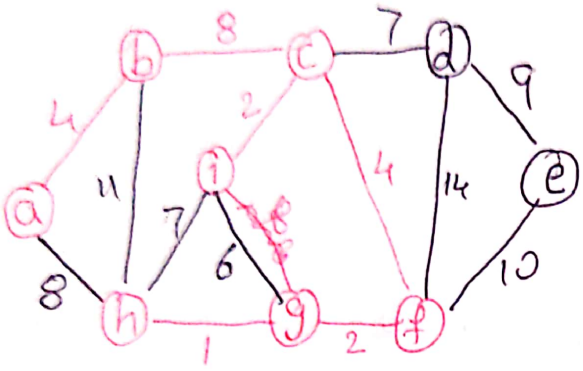
Key[e] = 10 π[e] = f

Q = {d, e, g, h}
7 10 2 8

V = {a, b, c, i, f}

Extract_min(Q) ⇒ g

(7)



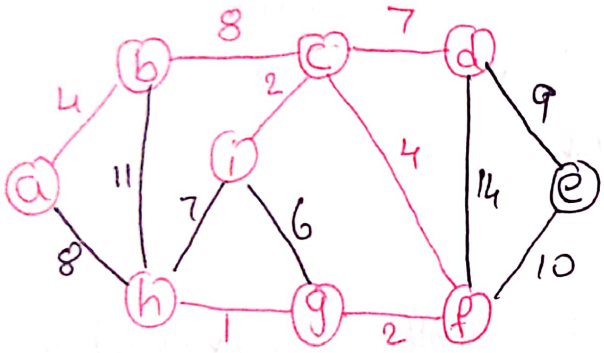
Key[h] = 1 π[h] = g

Q = {d, e, h}
7 10 1

V = {a, b, c, i, f, g}

Extract_min(Q) ⇒ h

(8)

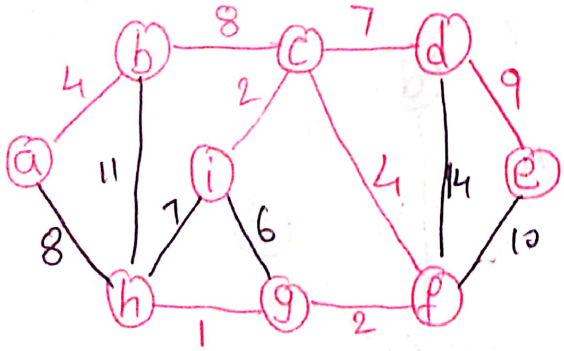


7 10
Q = {d, e}

V = {a, b, c, i, f, g, h}

Extract_min(Q) ⇒ d

(9)



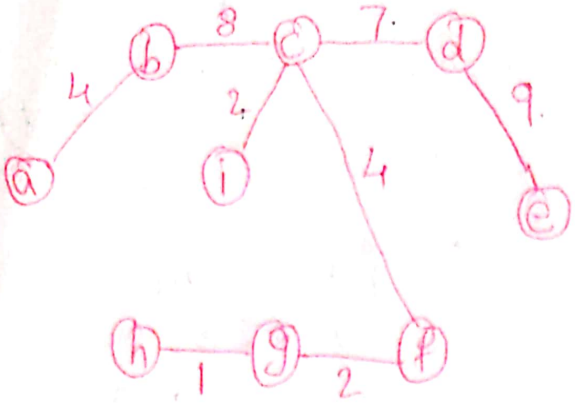
Key[e] = 9 π[e] = f

Q = {e}
9

V = {a, b, c, i, f, g, h, d}

Extract_min(Q) ⇒ e

Final MST



$Q = \emptyset$

~~$V = \{a, b, c, d, e$~~

$V = \{a, b, c, i, f, g, h, d, e\}$

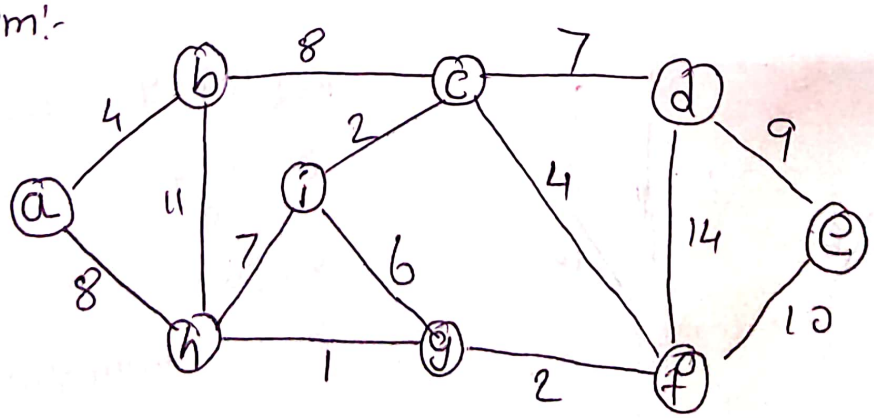
Total min cost = ~~37~~

Kruskal's Algorithm

$A \leftarrow \emptyset$
for each vertex $v \in V$
    do MAKE-SET(v)
sort E into increasing order by w
for each (u,v) taken from the sorted list
    do if FIND_SET(u) $\neq$ FIND_SET(v)
        then $A \leftarrow A \cup \{(u,v)\}$
            UNION(u,v)
return A

Time complexity of Kruskal's algorithm $O(V \log V)$ or $O(E \log E)$

Problem:-



descending order of their edges

1. (h, g)
2. (c, i), (g, f)
4. (a, b), (c, f)
6. (i, g)
7. (c, d), (i, h)
9. (a, h) (b, c)
10. (d, e)
12. (e, f)
13. (b, h)
14. (d, f)

The vertices are in different forest

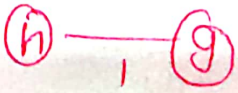
{a}, {b}, {c}, {d}, {e}, {f}, {g}, {h}, {i}

1. Add (h, g)

{g, h}, {a}, {b}, {c}, {d}, {e}, {f}, {i}

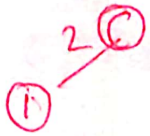
2. Add (c, i)

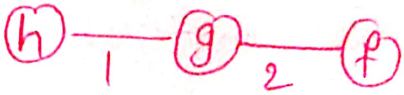
{g, h}, {c, i}, {a}, {b}, {d}, {e}, {f}



3. Add (g, f)

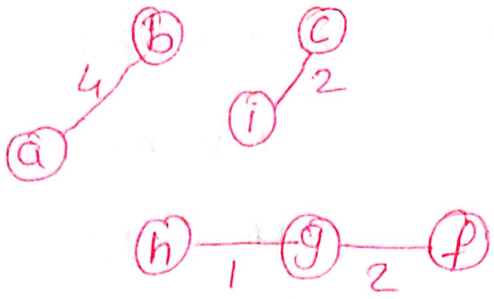
{g, h, f}, {c, i}, {a}, {b}, {d}, {e}



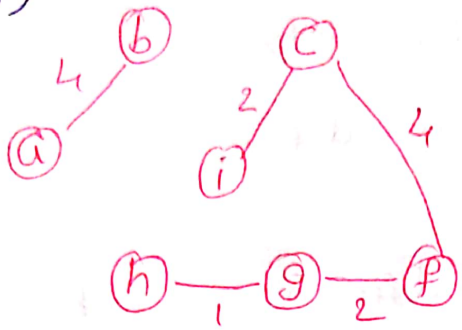


4. Add (a, b)

{g, h, f}, {c, i}, {a, b}
{d}, {}, {e}



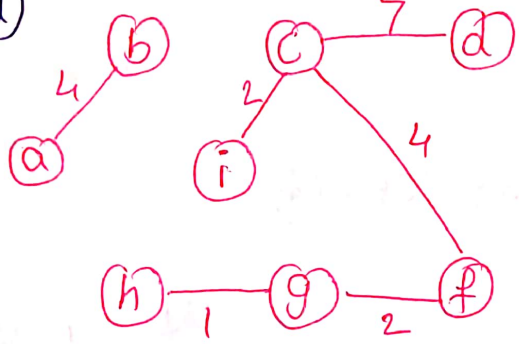
5 Add (c, f)



{g, h, f, c, i}, {a, b}, {d}, {e}

6. ignore (i, g) ∵ it form a cycle

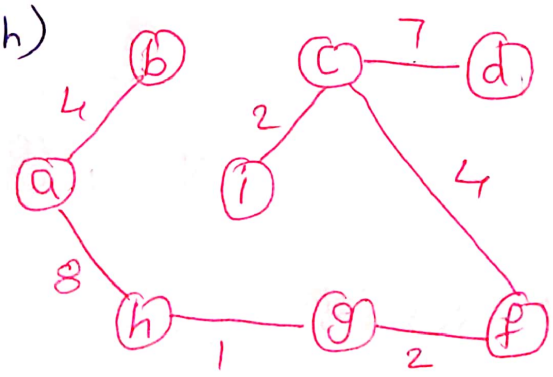
7. Add (c, d)



{g, h, f, c, i, d}, {a, b}, {e}

8. ignore (i, h) ∵ it forms a cycle

9. Add (a, h)

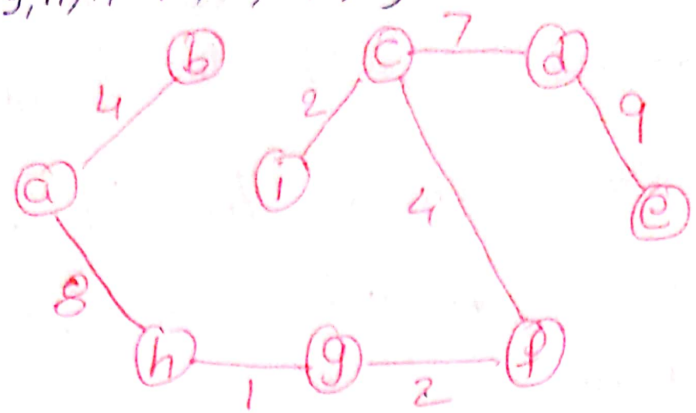


{g, h, f, c, i, d, a, b}, {e}

10. ignore (b, c) ∵ it forms a cycle

11. Add (d, e)

{g, h, f, c, i, d, a, b, e}



12. ignore (e, f) ∵ it forms a cycle

13. ignore (b, h) ∵ it forms a cycle

14. ignore (d, f) ∵ it forms a cycle

So, cost of MST is 37

Single source shortest Path:-

→ It is a greedy algorithm that solves the single-source shortest path problem for a directed graph $G=(V, E)$ with non-negative edges weights i.e $w(u, v) \geq 0$ for each edge $(u, v) \in E$

→ It is also called Dijkstra's algorithm. It maintain a set S of vertices whose final shortest-path weights from the source s have already been determined

→ That's for all vertices $v \in S$; we have $d[v] = \delta(s, v)$ The algorithm repeatedly selects the vertex $a \in V-S$

with minimum shortest-path estimate, insert u in to S and relaxes all edges leaving u

→ Because it always chooses the "lightest" or "closest" vertex in V − S to insert into set S, it is called as greedy

→ Dijkstra's algorithm finds shortest (minimum weight) path between a particular pair of vertices in a weighted directed graph with non-negative edge weights

→ Solve the "one vertex shortest path" problem

→ Create a table of information about the currently known best way to each each vertex and improve it until it reaches the best solution

Algorithm

```
Dijkstra(G, w, s)
Initialize-Single Source(G, s)
S = ∅
Q = G.V
while Q ≠ ∅
    u = Extract-min(Q)
    S = S ∪ {u}
    for each vertex v ∈ G.Adj[u]
        Relax(u, v, w)
```

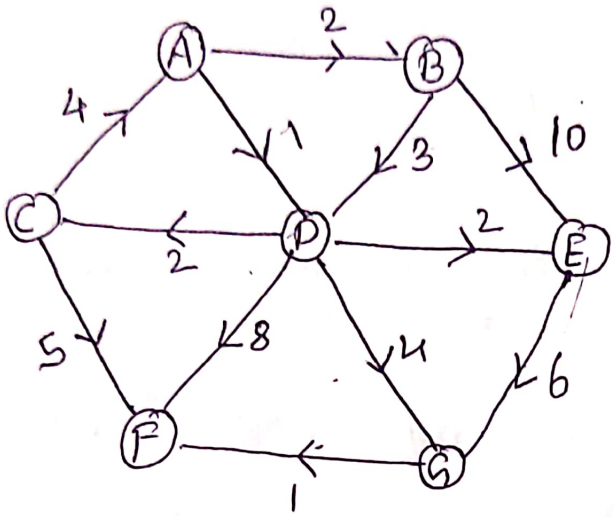
Time complexity of Dijkstra algorithm is $O(V^2)$

Relaxing edge (V, u)

if $d(u) > d(v) + c(v,u)$

$d(u) = d(v) + c(v,u)$

Problem:-



Let source vertex A

Sol:-

Source vertex - A

	B	C	D	E	F	G	
(D)	2	∞	(1)	∞	∞	∞	
B	(2)	3		3	9	5	
E		3		(3)	9	5	(Not updated)
C		(3)			9	5	(Not updated)
G					8	(5)	
F					(6)		

i. Initially Distance source = 0 and Distance (all vertices but source) = ∞ Pick vertex in List with minimum Distance

2. Distance (B) = 2 ; Distance (D) = 1

Pick ~~DIS~~ vertex in List with minimum Distance i.e D

3. Distance (C) = 1+2 = 3

Distance (E) = 1+2 = 3

Distance (F) = 1+8 = 9

Distance (A) = 1+4 = 5

4. Pick vertex ~~a~~ in list with minimum distance (B) and update neighbors

5. Distance of D and E is not updated ∵ D is already known and distance (E) ~~is~~ is larger than previously computed

6. Pick vertex List with minimum distance (E) and update neighbors and ~~no~~update from E.

7. Pick vertex List with minimum distanc (C) and update neighbors distance (F) = 3+5 = 8

8. Pick vertex with minimum distance (A) and update neighbors distance (F) = min (8, 5+1) = 6

Previous distance

9. Pick vertex not in S with lowest cost (F) and update neighbors

So final shortest path from source to all vertex is

~~Dist~~

A → B = 2 A → E = 3

A → C = 3 A → F = 6

A → D = 1 A → G = 5

Disadvantage of Dijkstra Algorithm:

→ It does a blind search, so wastes a lot of time while processing

→ It can't handle negative edges

→ It leads to the acyclic graph and most often cannot obtain the right shortest path

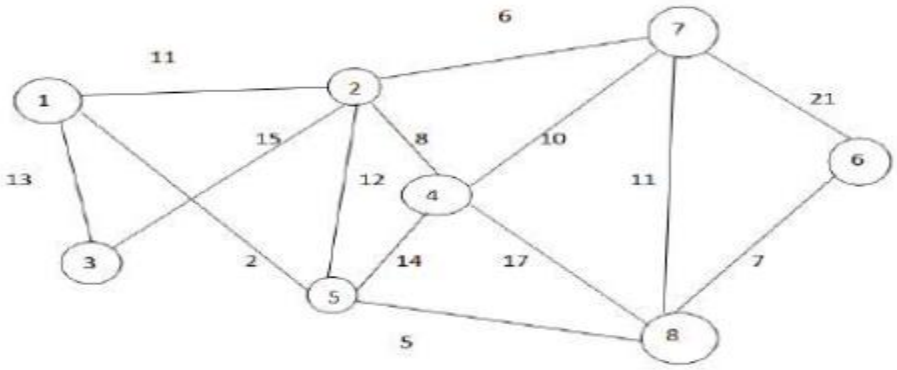
→ we need to keep track of vertices that have been visited.

Short Questions

1. State the Job – Sequencing Deadline Problem.
2. Define i) Principles of optimality ii) Feasible solution iii) Optimal solution
3. State the Greedy Knapsack Problem
4. Distinguish between Prim's and Kruskal's Spanning tree algorithm
5. Find an optimal solution to the knapsack instance n=4 objects and the capacity of knapsack m=15, profits (10, 5, 7, 11) and weight are (3, 4, 3, 5)
6. Distinguish between Divide and conquer and Greedy method
7. Write Control Abstraction of Greedy method
8. Define Minimum Cost Spanning tree and list its applications
9. What is the importance of knapsack algorithm in our daily life?
10. What is the time complexity of the Job sequencing with deadlines using greedy algorithm?
11. State the principle of optimality. Find two problems for which the principle does not hold.
12. State the principle of optimality. Find two problems for which the principle does not hold.
13. Discuss feasible solution, optimal solution and objective functions with example
14. Write about principle of optimality in shortest path problem
15. Explain about single source shortest path problem

Long Questions:

1. Explain the general principle of Greedy method and also list the applications of Greedy method.
2. State the Greedy Knapsack? Find an optimal solution to the Knapsack instance n=3, m=20, (P1, P2, P3) = (25, 24, 15) and (W1, W2, W3) = (18, 15, 10).
3. Find an optimal solution to the knapsack instance n=7 objects and the capacity of knapsack m=15. The profits and weights of the objects are (P1,P2,P3, P4, P5, P6, P7)= (10, 5,15,7,6,18,3) (W1,W2,W3,W4,W5,W6,W7)= (2,3,5,7,1,4,1)
4. What is a Spanning tree? Explain Prim's Minimum cost spanning tree algorithm with suitable example.
5. What is a Minimum Cost Spanning tree? Explain Kruskal's Minimum cost spanning tree algorithm with suitable example.
6. Explain differences between Prim's and Kruskal's Minimum spanning Tree algorithm. Derive the time complexity of Kruskal's algorithm.
7. What is the role of 'min' cost edge in the graph to find minimum cost spanning tree using Krushkal algorithm? Give the implementation.
8. Define spanning tree. Compute a minimum cost spanning tree for the graph of figure using prim's algorithm and kruskal's algorithm



9. Explain the greedy technique for solving the Job Sequencing problem.
10. How to insert more number of jobs in feasible solution set J={} to maximize the profit using greedy method? Explain algorithm.
11. State the Job – Sequencing with deadlines problem. Find an optimal sequence to the n=5 Jobs where profits (P1,P2,P3,P4,P5) = (20,15,10,5,1)and deadlines (d1,d2,d3,d4,d5) =(2,2,1,3,3).
12. Consider the following 5 jobs with their associated deadline and profit.

Index	1	2	3	4	5
job	j2	j1	j4	j3	j5
deadline	1	2	2	3	1
profit	100	60	40	20	20

Solve the problem to earn maximum profit when only one job can be scheduled or processed at any given time.

13. Use the greedy algorithm for sequencing unit time jobs with deadlines and profits to generate the solution when n=7,(p1,p2,…p7)=(3,5,20,18,1,6,30),and (d1,d2,…,d7)=(1,3,4,3,2,1,2)
14. Derive time complexity of job sequencing with deadlines .Obtain the optimal solution when n=5, (p1, p2,…)=(20,15,10,5,1) and (d1,d2,…)=(2,2,1,3,3).
15. Discuss the single – source shortest paths algorithm with suitable example.
16. Discuss the Dijkstra's single source shortest path algorithm and derive the time complexity of this algorithm.
17. Generate the actions of shortest paths for the given graph from vertex 1 to all remaining vertices. 1→2=20, 2→1=2, 1→3=15, 2→5=10, 2→6=30, 3→6=10, 3→4=4, 5→4=15, 6→4=4, 6→5=10.